claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_793a5d86-1c4\agent-a833a6e89049fa40f.jsonl`
- SHA-256: `879bc470f6f38bd3f4089d29a8593aa87a5ee65742a1c85741630cad833d2654`
- Source modified: `2026-07-02T08:18:24+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_793a5d86-1c4`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`

Transcript

1. user (2026-07-02T08:16:24.736Z)

You are reviewing OPEN GitHub PR #417 on Khelsutra/badminton-highlight-indexer to decide its fate: should we TAKE IT OVER (adopt/rebase/verify/merge) or CLOSE it as superseded by already-merged work?

PR #417: "fix(compute): rescue-or-cancel on Cloud Run bucket-poll timeout (R1-21)" (head branch: origin/fix/cloud-run-poll-timeout-cancel, mergeable=MERGEABLE)
Investigation hint: MERGEABLE CLEAN. On PolicyTimeout: one final done-marker check to rescue a late worker, else cancel the still-running Cloud Run execution. CHECK: does master backend/compute (CloudRunCompute.run_infer) still let PolicyTimeout propagate without cancelling the execution? If unchanged -> real still-needed work (take over clean).

Already MERGED onto master (verify against actual master code, do not assume):
- #422 = fix for --output-dir clobbering config.json output.output_dir (CODE_MAP updated for it)
- #423 = docs: health-remediation plan added + DOC_STATUS/README/NEXT_STEPS registered; audit marked snapshot
- #424 = P2: /api/process idempotent by content hash (#340 re-bill fix)
- #425 = P3: /api/source resolves remote source into output-scoped source_cache, reuses across Range requests, warms annotation proxy
- #426 = P4: bounded ffprobe/ffmpeg subprocess timeouts on serving paths
- #427 = P5: clean remote-input scratch dirs after process/compile/assets/CLI; durable reports
- #428 = P1/P6: backend/storage/gcs.py lazy-import mypy typing fix
master HEAD = d00b4db "fix gcs storage lazy import typing (#428)"

Working dir: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (currently on master @ d00b4db). This is a SHARED checkout used by sibling sessions ? DO NOT checkout branches, DO NOT modify the working tree, DO NOT run git merge/rebase/add/commit. Read-only investigation only.

Do this:
1. Get the PR's intended change: run gh pr diff 417 -R Khelsutra/badminton-highlight-indexer (read the full diff). Also gh pr view 417 -R Khelsutra/badminton-highlight-indexer --json body,files,statusCheckRollup if useful.
2. Read the CURRENT master version of every file the PR touches (Read / Grep the files under C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer). To see the PR's branch version of a file without checkout, use: git -C "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer" show origin/fix/cloud-run-poll-timeout-cancel:<path>.
3. Determine whether master ALREADY implements the PR's intent (superseded) or still lacks it. Cite concrete master file:line evidence either way. Do not trust the hint or handoff ? verify in code.
4. If CONFLICTING, explain precisely why (which files/hunks) and whether the conflict is because master already did the same thing (=> superseded/close) or because of adjacent edits (=> rebasable).
5. Judge correctness/quality of the PR change itself.

```
6. Recommend a fate:
   - CLOSE_SUPERSEDED: master already implements the intent; the PR adds nothing material.
   - TAKE_OVER_CLEAN: mergeable & still-needed; adopt and verify as-is.
   - TAKE_OVER_REBASE: still-needed but conflicting; needs a rebase onto master + re-verify,
then merge.
   - OWNER_DECISION: product/strategy/ambiguous ? needs the owner.

Return ONLY the structured object. Be precise and cite master file:line in master_evidence.
```

2. assistant (2026-07-02T08:16:27.628Z)

```
I'll investigate PR #417 to decide its fate. Let me start by getting the PR's intended change
and examining the current master code.
```

3. user (2026-07-02T08:16:31.028Z)

```
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index ddfc085..598d62f 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -7,7 +7,10 @@
   2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16
? the
      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API status,
so
      it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero (D7):
the
-     cold-start is absorbed by the async poll.
+     cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can undercut
the
+     job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
healthy
+     worker that finished just past the deadline is rescued, not wasted ? then best-effort
CANCELS
+     the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
   3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
      ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -33,6 +36,7 @@
 from backend.infrastructure.execution_policy import (
     DEFAULT_POLICY,
     ExecutionPolicy,
+    PolicyTimeout,
     poll_until,
 )
 from backend.storage import fetch, get_storage, parse_uri
@@ -65,6 +69,13 @@ def run_job(
         """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``; return
an
         execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""

+    @abstractmethod
+    def cancel_execution(self, execution: str) -> None:
+        """Cancel a running execution by the name ``run_job`` returned. Called when the bucket-
poll
+        times out so the abandoned execution doesn't keep billing the GPU until ``--task-
timeout``.
+        May raise ? the caller treats every failure as best-effort (the original poll timeout
is
+        NEVER masked by a cancel error)."""
+

 class GoogleCloudRunJobClient(CloudRunJobClient):
     """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI
```

```
uses
@@ -111,6 +122,29 @@ def run_job(
             or "execution"
         )

+    def cancel_execution(self, execution: str) -> None:
+        # run_job falls back to the literal string "execution" when the operation metadata
carries
+        # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK import
+        # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI (where
+        # google-cloud-run is absent).
+        if "/executions/" not in (execution or ""):
+            raise RuntimeError(
+                f"not a fully-qualified Cloud Run execution name: {execution!r} ? expected "
+                "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not resolve
"
+                "one from the operation metadata, so there is nothing to cancel by name)."
+            )
+        try:
+            from google.cloud import run_v2  # type: ignore[attr-defined]
+        except Exception as exc:  # pragma: no cover - real SDK not present in CI
+            raise RuntimeError(
+                "google-cloud-run is required to cancel a Cloud Run execution; install it on
the "
+                "control plane (it is intentionally NOT a CI/test dependency)."
+            ) from exc
+        client = (
+            run_v2.ExecutionsClient()
+        )  # pragma: no cover - exercised only on a real run
+        client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
+

 class CloudRunCompute(ComputeBackend):
     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
@@ -170,12 +204,15 @@ def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
             done_uri,
         )

-        marker = poll_until(
-            lambda: self._read_marker(done_uri),
-            self._policy,
-            label="cloud-run-infer",
-            logger=LOG,
-        )
+        try:
+            marker = poll_until(
+                lambda: self._read_marker(done_uri),
+                self._policy,
+                label="cloud-run-infer",
+                logger=LOG,
+            )
+        except PolicyTimeout as timeout_exc:
+            marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
         video_id = str(marker.get("video_id", ""))
         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
         # garbled marker must not masquerade as rc=0. (The worker always writes both fields.)
@@ -200,6 +237,53 @@ def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
             execution=str(execution or ""),
         )

+    def _handle_poll_timeout(
+        self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
+    ) -> dict:
+        """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
```

```
+
+        The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed job's own
+        ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past the
deadline:
+        ONE final marker check rescues that completed GPU run instead of wasting it. If the
marker
+        is genuinely absent, the execution is still running ? best-effort cancel it (otherwise
the
+        L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout`` carrying
the
+        execution name so the operator can verify the state in the GCP console. A cancel
failure
+        NEVER masks the original timeout."""
+        try:
+            late = self._read_marker(done_uri)
+        except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence ?
failure
+            LOG.warning(
+                "cloud-run: final post-timeout marker check failed ? treating as absent",
+                exc_info=True,
+            )
+            late = None
+        if late is not None:
+            LOG.warning(
+                "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check "
+                "? rescuing the completed result",
+                self._job_name,
+                execution,
+            )
+            return late
+        try:
+            self._client.cancel_execution(execution)
+        except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the timeout
+            cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
+            LOG.warning(
+                "cloud-run: best-effort cancel of execution=%s failed: %s",
+                execution,
+                cancel_exc,
+                exc_info=True,
+            )
+        else:
+            cancel_note = "cancel requested"
+            LOG.info("cloud-run: requested cancel of execution=%s", execution)
+        raise PolicyTimeout(
+            f"{timeout_exc} [cloud-run job={self._job_name} "
+            f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution "
+            "state in the GCP console]"
+        ) from timeout_exc
+
     # --- storage-backed completion signal (bucket-poll) ---
     def _read_marker(self, done_uri: str) -> Optional[dict]:
         """The done-marker dict if the worker has written it, else None ? poll_until keeps
waiting."""
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 5e2e46f..53923a9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -88,6 +88,11 @@ def run_job(self, job_name, argv, *, region, project=None):
             )
         return "exec-1"

+    def cancel_execution(self, execution):
+        raise AssertionError(
+            "cancel must never fire on the happy path"
+        )  # pragma: no cover
```

```
+

 @pytest.fixture
 def cloud_env(tmp_path, monkeypatch):
diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
index 2955192..61d82fe 100644
--- a/tests/test_compute_backend.py
+++ b/tests/test_compute_backend.py
@@ -15,7 +15,7 @@
 from backend.compute import ComputeJobSpec, get_compute_backend
 from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
 from backend.config import AppConfig
-from backend.infrastructure.execution_policy import ExecutionPolicy
+from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout

 # A fast, no-wait policy (the fake client writes the marker synchronously, so the first poll
hits).
 _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
@@ -63,6 +63,11 @@ def run_job(self, job_name, argv, *, region, project=None):
             )
         return "exec-123"

+    def cancel_execution(self, execution):
+        raise AssertionError(
+            "cancel must never fire on the happy path"
+        )  # pragma: no cover
+

 class _NoopRunClient(CloudRunJobClient):
     """Records the dispatch but does NO filesystem writes ? for tests that fake the storage
layer
@@ -70,6 +75,7 @@ class _NoopRunClient(CloudRunJobClient):

     def __init__(self):
         self.calls: list[dict] = []
+        self.cancelled: list[str] = []

     def run_job(self, job_name, argv, *, region, project=None):
         self.calls.append(
@@ -77,6 +83,32 @@ def run_job(self, job_name, argv, *, region, project=None):
         )
         return "exec"

+    def cancel_execution(self, execution):
+        self.cancelled.append(execution)
+
+
+class _NeverDoneClient(CloudRunJobClient):
+    """Simulates a still-running execution: dispatch succeeds (returning a fully-qualified
+    execution name) but NO done-marker is ever written, so the bucket-poll must time out."""
+
+    EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-slow"
+
+    def __init__(self, *, cancel_error: Exception | None = None):
+        self.calls: list[dict] = []
+        self.cancelled: list[str] = []
+        self._cancel_error = cancel_error
+
+    def run_job(self, job_name, argv, *, region, project=None):
+        self.calls.append(
+            {"job": job_name, "argv": list(argv), "region": region, "project": project}
+        )
+        return self.EXEC_NAME
+
+    def cancel_execution(self, execution):
```

```
+        self.cancelled.append(execution)
+        if self._cancel_error is not None:
+            raise self._cancel_error
+

 # ------------------------------------------------------------------ factory (default-OFF)

@@ -120,6 +152,19 @@ def test_real_client_rejects_missing_project():
             )


+def test_real_client_cancel_rejects_unresolvable_execution_name():
+    """cancel_execution must fail loudly on a non-resolvable execution name ? run_job falls
back
+    to the literal "execution" when the operation metadata carries no name, and cancelling that
+    would be a bogus API call. The guard runs BEFORE the google-cloud-run import (mirrors the
+    project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
+    from backend.compute.cloud_run import GoogleCloudRunJobClient
+
+    client = GoogleCloudRunJobClient()
+    for bogus in ("", "execution", "exec-123"):
+        with pytest.raises(RuntimeError, match="fully-qualified"):
+            client.cancel_execution(bogus)
+
+
 # ------------------------------------------------------------ CloudRunCompute dispatch + poll


@@ -267,6 +312,92 @@ def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
     )  # >=1 absent poll occurred before the marker appeared (loop ran)


+# ---------------------------------------- poll timeout: rescue-or-cancel (R1-21 GPU cost
leak)
+
+# max_wait_sec=0.0 ? poll_until gives up after its FIRST absent check (no real waiting).
+_TIMEOUT_NOW = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=0.0)
+
+
+def test_poll_timeout_cancels_execution_and_raises(tmp_path):
+    """Poll deadline + no marker ? the still-running execution is cancelled (with the EXACT
name
+    run_job returned) and PolicyTimeout surfaces carrying that name for the operator."""
+    client = _NeverDoneClient()
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_TIMEOUT_NOW,
+        token="tok",
+    )
+    with pytest.raises(PolicyTimeout) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # cancel got the right name
+    msg = str(exc.value)
+    assert _NeverDoneClient.EXEC_NAME in msg  # operator can find the execution in GCP
+    assert "cancel requested" in msg
+    assert "max_wait_sec" in msg  # the original timeout text is preserved, not masked
+
+
+def test_poll_timeout_cancel_failure_still_raises_policy_timeout(tmp_path):
+    """Best-effort means BEST-EFFORT: a cancel API failure must never mask the original timeout
```

```
?
+    the caller (main.py's dispatch handler) still sees PolicyTimeout, with the failure
noted."""
+    client = _NeverDoneClient(cancel_error=RuntimeError("cancel API unreachable"))
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_TIMEOUT_NOW,
+        token="tok",
+    )
+    with pytest.raises(PolicyTimeout) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # the attempt WAS made
+    msg = str(exc.value)
+    assert _NeverDoneClient.EXEC_NAME in msg
+    assert "cancel API unreachable" in msg  # the cancel failure is surfaced, not swallowed
+    assert "max_wait_sec" in msg  # ...alongside the original timeout text
+
+
+def test_poll_timeout_late_marker_rescues_result(monkeypatch):
+    """A slow-but-healthy worker that finishes JUST past the poll deadline (max_wait_sec=1800
can
+    undercut the job's --task-timeout=3600) is rescued by the ONE final post-timeout marker
check:
+    the completed GPU result is ingested, nothing is cancelled, no timeout surfaces."""
+    from backend.compute import cloud_run as cr
+
+    exists_calls = {"n": 0}
+
+    class _FakeStore:
+        def exists(self, ref):
+            exists_calls["n"] += 1
+            return exists_calls["n"] >= 2  # absent for the poll, present on the final check
+
+    monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
+
+    def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+        if dst is None:
+            dst = os.path.join(tempfile.mkdtemp(prefix="cr-late-"), "m.json")
+        with open(dst, "w", encoding="utf-8") as f:
+            f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
+        return dst
+
+    monkeypatch.setattr(cr, "fetch", fake_fetch)
+    client = _NoopRunClient()
+    backend = cr.CloudRunCompute(
+        run_client=client, job_name="j", region="r", token="t", policy=_TIMEOUT_NOW
+    )
+    result = backend.run_infer(
+        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
+    )
+    assert result.returncode == 0
+    assert result.video_id == "vlate"  # the late result is ingested, not wasted
+    assert client.cancelled == []  # a finished execution is never cancelled
+    assert exists_calls["n"] == 2  # exactly ONE extra post-timeout check
+
+
 # ---------------------------------------------------- worker wiring: dispatch branch + done-
marker
```

4. user (2026-07-02T08:16:32.562Z)

{"body":"Fixes audit finding **R1-21** (AUDIT_REPORT_khelsutra-guru_2026-07-02): a bucket-poll timeout abandons a still-running L4 execution.\n\n## Summary\n\n**What:**
`CloudRunCompute.run_infer` blocked on `poll_until(...)`; when the poll deadline (`max_wait_sec`, default 1800s) hit, `PolicyTimeout` propagated straight to `main.py`'s dispatch handler, which marked the local job terminal-failed. Nothing cancelled the still-running Cloud Run execution (it kept billing the L4 until the job's `--task-timeout 3600`), and because the poll budget undercuts the task budget, a slow-but-healthy 30-60 min inference *systematically* ended as a local \"cloud dispatch error\" while its late done-marker + outputs were silently wasted.\n\n**How (on `PolicyTimeout`):**\n- **ONE final done-marker check** ? a worker that finished just past the poll deadline is rescued: its completed result is ingested normally (no cancel, no error). The final check is itself best-effort (a storage error there is treated as marker-absent, never masking the timeout).\n- **Best-effort cancel** ? new `CloudRunJobClient.cancel_execution(execution)` abstract method (the injected test seam). The real `GoogleCloudRunJobClient` validates the stored execution name is fully-qualified (`.../executions/...`) **before** the lazy `google-cloud-run` import ? mirroring the #342 project guard so the guard is CI-exercisable ? then calls `run_v2.ExecutionsClient().cancel_execution`. (`run_job` falls back to the literal string `\"execution\"` when operation metadata carries no name; cancelling that would be a bogus API call, so it fails loudly instead.)\n- **Honest failure message** ? the re-raised `PolicyTimeout` carries the job name, the execution name, and the cancel outcome (\"cancel requested\" / \"best-effort cancel FAILED (?)\") so the operator can verify the execution state in the GCP console. A cancel failure **never** masks the original timeout (exception type stays `PolicyTimeout`, chained `from` the original).\n\nBehavior-preserving otherwise: the happy path (marker arrives in budget) is byte-identical; the timeout path previously raised `PolicyTimeout` and still does.\n\n**Not in this PR (deliberately):** retuning `max_wait_sec` vs the deployed job's `--task-timeout` (a live-job policy change ? owner-gated per the M7 real-run convention), and the real-client cancel path itself is owner-verified on a real run (the SDK is intentionally not a CI dependency ? same stance as `run_job`).\n\n## Verification\n\nRan locally in the worktree (Windows, Python 3.13):\n- `python -m pytest tests/test_compute_backend.py tests/test_api_cloud_dispatch.py -q` ? **42 passed** (includes 4 new tests: cancel called with the exact execution name; cancel failure still surfaces `PolicyTimeout` with both errors in the message; late done-marker rescues the result with zero cancels; real client rejects unresolvable execution names before the SDK import). Happy-path fakes now `raise AssertionError` from `cancel_execution`, locking in that cancel never fires outside the timeout path.\n- `ruff check backend/compute/cloud_run.py tests/test_compute_backend.py tests/test_api_cloud_dispatch.py` ? clean.\n- `mypy backend/compute/cloud_run.py` ? no new errors (the one reported error, `backend/storage/gcs.py:31 google.cloud has no attribute \"storage\"`, reproduces identically at base `e410136` with my change stashed ? a local-env artifact of partially-installed google namespace packages; CI's `mypy backend training` env resolves it via `ignore_missing_imports`).\n- **Not verified locally:** the real `GoogleCloudRunJobClient.cancel_execution` against live GCP (no cloud spend from CI/dev; owner-verified on the next M7-style real run).\n\nCI covers: full suite via `run_all_tests.py` (offline/mocked) + coverage floor, `ruff check .`, `mypy backend training`, link-check.\n\n## Checks\n- [x] Targeted tests green; `ruff` clean on changed files; `mypy` no new errors; coverage not lowered (new branches are all test-covered). Full suite: CI.\n- [x] No markdown changes ? link-check unaffected.\n\n? Generated with [Claude Code](https://claude.com/claude-code)","files":[{"path":"backend/compute/cloud_run.py","additions":91,"deletions":7,"changeType":"MODIFIED"},{"path":"tests/test_api_cloud_dispatch.py","additions":5,"deletions":0,"changeType":"MODIFIED"},{"path":"tests/test_compute_backend.py","additions":132,"deletions":1,"changeType":"MODIFIED"}],"headRefName":"fix/cloud-run-poll-timeout-cancel","mergeable":"MERGEABLE","statusCheckRollup":[{"__typename":"CheckRun","completedAt":"2026-07-02T04:55:27Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566413950/job/84694528171","name":"Ruff lint","startedAt":"2026-07-02T04:55:18Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:55:28Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566413950/job/84694528184","name":"Leak scan (PII / attribution guard)","startedAt":"2026-07-02T04:55:19Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:55:25Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566413950/job/84694528203","name":"Markdown link check (live docs)","startedAt":"2026-07-02T04:55:18Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"

2026-07-
02T04:55:44Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528182","name":"Mypy (lenient baseline)","st
artedAt":"2026-07-
02T04:55:18Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-07-
02T04:55:39Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528235","name":"Syntax sweep + import smoke
(no GPU stack)","startedAt":"2026-07-
02T04:55:19Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-07-
02T04:58:05Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528175","name":"Full test suite (offline, mo
cked)","startedAt":"2026-07-
02T04:55:18Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-07-
02T04:55:39Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528193","name":"Golden fixtures cross-OS
(ubuntu-latest)","startedAt":"2026-07-
02T04:55:18Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-07-
02T04:56:15Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528177","name":"Golden fixtures cross-OS
(windows-latest)","startedAt":"2026-07-
02T04:55:19Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-07-
02T04:55:35Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528207","name":"Golden fixtures cross-OS
(macos-latest)","startedAt":"2026-07-
02T04:55:18Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-07-
02T04:55:51Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528191","name":"Dependency license scan (no
copyleft in runtime deps)","startedAt":"2026-07-
02T04:55:19Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-07-
02T04:56:24Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28566413950/job/84694528204","name":"Built wheel smoke (install +
launch)","startedAt":"2026-07-02T04:55:18Z","status":"COMPLETED","workflowName":"CI"}]}

5. assistant (2026-07-02T08:16:36.383Z)

Now let me read the current master version of the file the PR touches, focusing on `run_infer` and the `CloudRunJobClient`/`CloudRunCompute` classes.

6. user (2026-07-02T08:16:37.868Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
```

```
13
14      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19      owner-verified on the M7 real run.
20      """
21
22      from __future__ import annotations
23
24      import json
25      import logging
26      import os
27      import tempfile
28      import uuid
29      from abc import ABC, abstractmethod
30      from typing import Any, List, Optional
31
32      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33      from backend.infrastructure.execution_policy import (
34          DEFAULT_POLICY,
35          ExecutionPolicy,
36          poll_until,
37      )
38      from backend.storage import fetch, get_storage, parse_uri
39
40      LOG = logging.getLogger("compute.cloud_run")
41
42      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
43      # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
44      _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
45          "deployment.compute_target=local_gpu",
46          "indexing.detector_impl=native",
47      )
48
49
50      class CloudRunJobClient(ABC):
51          """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
52
53          Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
54          overrides the job's container ``args`` for this execution and starts it."""
55
56          @abstractmethod
57          def run_job(
58              self,
59              job_name: str,
60              argv: List[str],
61              *,
62              region: str,
63              project: Optional[str] = None,
64          ) -> str:
65              """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
66              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
67
```

```

    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""

        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
            # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
            if not project:
                raise RuntimeError(
                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
                    "to resolve the job path; set it on the control plane (see
get_compute_backend)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.JobsClient()
            )  # pragma: no cover - exercised only on the real M7 run
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
```

```
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            marker = poll_until(
                lambda: self._read_marker(done_uri),
                self._policy,
                label="cloud-run-infer",
                logger=LOG,
            )
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
```

```
186                     "video_id) ? treating as failure",
187                     self._job_name,
188                 )
189                 rc = rc or 1
190             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
191             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
192             LOG.info(
193                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
194             )
195             return ComputeResult(
196                 returncode=rc,
197                 outputs_uri=outputs_uri,
198                 video_id=video_id,
199                 report=report,
200                 execution=str(execution or ""),
201             )
202 
203         # --- storage-backed completion signal (bucket-poll) ---
204         def _read_marker(self, done_uri: str) -> Optional[dict]:
205             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206             ref = parse_uri(done_uri)
207             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208                 return None
209             return (
210                 self._read_json(done_uri) or {}
211             )  # empty dict still terminates the poll (present == done)
212 
213         def _read_json(self, uri: str) -> dict:
214             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
215             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
216             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
217             try:
218                 ref = parse_uri(uri)
219                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
220                     dst = os.path.join(td, ref.name or "obj.json")
221                     local = fetch(uri, dst, config=self._storage_config)
222                     with open(local, "r", encoding="utf-8") as f:
223                         data: Any = json.load(f)
224                 return data if isinstance(data, dict) else {}
225             except Exception:
226                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
227                 return {}
228 
```

7. user (2026-07-02T08:16:39.706Z)

```
d00b4db fix gcs storage lazy import typing (#428)
---
d00b4db7ab11b61df356940d2a6fc77486afe50c
--- branch ---
master
```

8. assistant (2026-07-02T08:16:47.442Z)

The master version of `backend/compute/cloud_run.py` is clear. Let me confirm the key facts:

Master state (verified in code at `d00b4db`):
- **No `PolicyTimeout` import** ? line 33-37 imports only `DEFAULT_POLICY, ExecutionPolicy, poll_until`. The PR adds `PolicyTimeout` to this import.
- **No `cancel_execution` abstract method** ? `CloudRunJobClient` (lines 50-66) has only `run_job`. The PR adds the abstract `cancel_execution`.

- **No cancel implementation** in `GoogleCloudRunJobClient` (lines 69-112) ? only `run_job`. The PR adds it.
- **`run_infer` (lines 173-178)** calls `poll_until(...)` with **no try/except** around it ? a `PolicyTimeout` propagates straight up, uncaught. The PR wraps it in try/except and delegates to `_handle_poll_timeout`.
- **No `_handle_poll_timeout` method** exists in master. The PR adds it.

This confirms master does NOT implement the PR's intent. Let me verify the `PolicyTimeout` symbol exists in the execution_policy module (so the PR import is valid), and check that no sibling merge introduced any adjacent edits to this file since the PR branch was cut.

9. user (2026-07-02T08:16:54.549Z)

```
=== PolicyTimeout defined? ===
41:class PolicyTimeout(TimeoutError):
=== merge-base of PR branch vs master ===
e4101365cb5f3a608e76f8baf1d2e0d65ce1738e
=== master HEAD ===
d00b4db7ab11b61df356940d2a6fc77486afe50c
=== does master==merge-base? (clean rebase-free) ===
=== files changed on master since merge-base touching cloud_run.py ===
19ebe5a clean remote input scratch dirs (#427)
```

10. assistant (2026-07-02T08:16:59.214Z)

There's one commit on master (`19ebe5a`, PR #427) that touched `cloud_run.py` after the PR branch's merge-base. GitHub reports MERGEABLE, but let me verify what #427 changed in this file and whether it overlaps with the PR's hunks ? that determines whether the merge is truly clean or just auto-mergeable at the git level.

11. user (2026-07-02T08:17:01.280Z)

```
=== What #427 (19ebe5a) changed in cloud_run.py ===
commit 19ebe5a5da34a23e71b8eaf3980bfc5f4554208b
Author: Avi <avi.dullu@gmail.com>
Date:   Thu Jul 2 13:31:33 2026 +0530

    clean remote input scratch dirs (#427)

diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index ddfc085..1d4143f 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -216,12 +216,11 @@ class CloudRunCompute(ComputeBackend):
         # would silently return {} for every gs://|s3:// object (the only backends this shim
targets).
         try:
             ref = parse_uri(uri)
-            dst = os.path.join(
-                tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
-            )
-            local = fetch(uri, dst, config=self._storage_config)
-            with open(local, "r", encoding="utf-8") as f:
-                data: Any = json.load(f)
+            with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
+                dst = os.path.join(td, ref.name or "obj.json")
+                local = fetch(uri, dst, config=self._storage_config)
+                with open(local, "r", encoding="utf-8") as f:
+                    data: Any = json.load(f)
             return data if isinstance(data, dict) else {}
         except Exception:
             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
=== END ===
```

12. assistant (2026-07-02T08:17:08.563Z)

This is the key finding. PR #427 (merged, `19ebe5a`) modified `_read_json` (lines ~216-224) ? converting `tempfile.mkdtemp` to a `TemporaryDirectory` context manager.

Now, the PR #417 diff's context header shows its `_read_json` context as the OLD form (`with tempfile.TemporaryDirectory(prefix="cr-read-") as td:` ? wait, let me re-check). Actually looking closer at the PR #417 diff, its final hunk context (the `_read_marker` region around line 203) does NOT touch `_read_json`. Let me verify precisely which lines each touches and whether they conflict.

PR #417's hunks touch:
- Module docstring (lines ~7-10)
- Imports (line ~33)
- `run_job` abstractmethod region ? adds `cancel_execution` abstract (line ~65)
- `GoogleCloudRunJobClient` ? adds `cancel_execution` impl (line ~111-122)
- `run_infer` poll_until block (lines ~170-204)
- Adds `_handle_poll_timeout` (after line 200, before `_read_marker` at line ~203)

PR #427 touched only `_read_json` (lines ~216-224), which is **below** all of PR #417's hunks. Let me do an actual merge simulation to be certain there's no textual conflict.

13. user (2026-07-02T08:17:14.227Z)

```
=== Simulate merge of PR branch into master for cloud_run.py (read-only, no worktree change) ===
changed in both
  base   100644 ddfc08544f9f55a4182318557be1ef8d35549052 backend/compute/cloud_run.py
  our    100644 1d4143ff3afbabd89d38f0e8086bf73b44944eb2 backend/compute/cloud_run.py
  their  100644 598d62fe287d4378fdeead7a6e0f48497c9774c4 backend/compute/cloud_run.py
--
changed in both
  base   100644 2955192b144885c93dbf338e9d244c834b0e6754 tests/test_compute_backend.py
  our    100644 6c2f36b5231978df767c4fcab6d8d14cace22aeb tests/test_compute_backend.py
  their  100644 61d82fea2ef7dda2e94b9afeb27ea758bf9d641d tests/test_compute_backend.py
=== exit-grep done (empty above = no conflict markers) ===

=== Newer git merge-tree --write-tree form ===
bf5fc4278642e520fc8cef0470e67d57966c2499
--- conflict lines in output ---
=== done ===
```

14. assistant (2026-07-02T08:17:19.202Z)

The `git merge-tree --write-tree` (the modern 3-way merge form) produced a clean tree object (`bf5fc42...`) with **NO conflict lines**. The "changed in both" from the old `merge-tree` form just means both sides modified the file ? but the actual 3-way merge resolves cleanly because the hunks don't overlap (PR #417 touches everything above `_read_json`; PR #427 touched only `_read_json`).

This confirms GitHub's `MERGEABLE` verdict. Let me do a final confirmation that the merged result actually contains both changes coherently (the `_handle_poll_timeout` from #417 AND the `TemporaryDirectory` from #427), and verify the PR branch's own `_read_json` already incorporates #427's change or not.

15. user (2026-07-02T08:17:22.953Z)

```
=== PR branch's _read_json (does it have #427's TemporaryDirectory already?) ===
            )
        except PolicyTimeout as timeout_exc:
            marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
        video_id = str(marker.get("video_id", ""))
        # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-unreadable
/
        # garbled marker must not masquerade as rc=0. (The worker always writes both fields.)
```

```
        rc = int(marker.get("returncode", 1))
        if not video_id:
            LOG.warning(
                "cloud-run: job=%s completion marker present but empty/unreadable (no "
                "video_id) ? treating as failure",
                self._job_name,
            )
            rc = rc or 1
        outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
        report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
        LOG.info(
            "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc

=== Merged tree's cloud_run.py ? confirm BOTH _handle_poll_timeout and TemporaryDirectory
present ===
39:    PolicyTimeout,
73:    def cancel_execution(self, execution: str) -> None:
125:    def cancel_execution(self, execution: str) -> None:
214:        except PolicyTimeout as timeout_exc:
215:            marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
240:    def _handle_poll_timeout(
241:        self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
249:        L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout`` carrying
the
281:        raise PolicyTimeout(
303:            with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
```

16. assistant (2026-07-02T08:17:29.121Z)

The merged tree at line 303 contains #427's `TemporaryDirectory` AND at lines 240/281 contains #417's `_handle_poll_timeout`/`PolicyTimeout` ? both coexist coherently. Interesting: the PR branch's own `_read_json` (shown from `git show origin/fix/...`) already uses `TemporaryDirectory` form ? meaning the PR branch was cut from a base that already had #427's change (the diff's context lines showed `with tempfile.TemporaryDirectory` as unchanged context). So there is genuinely no overlap.

Now let me verify the interaction with `main.py`'s dispatch handler (the PR body claims the caller marks the job terminal-failed on `PolicyTimeout` ? I should confirm the caller still catches `PolicyTimeout` / a broad exception so the re-raised one is handled) and check the test files' current master state for adjacent-edit conflicts.

17. user (2026-07-02T08:17:32.964Z)

```
=== main.py: how is run_infer / PolicyTimeout handled around cloud dispatch? ===
=== search whole backend if not in main.py ===
backend/compute/base.py
backend/compute/cloud_run.py
backend/compute/__pycache__/base.cpython-313.pyc
backend/compute/__pycache__/cloud_run.cpython-313.pyc
backend/main.py
backend/worker/__main__.py
backend/worker/__pycache__/__main__.cpython-313.pyc
backend/__pycache__/main.cpython-313.pyc
```

18. user (2026-07-02T08:17:37.012Z)

```
215-            with urllib.request.urlopen(health_url, timeout=2) as r:
216-                if getattr(r, "status", 200) == 200:
217-                    break
218:        except Exception:
219-            _time.sleep(0.3)
220-    else:
221-        return False
222-    try:
```

```
223-        webbrowser.open(url)
224:    except Exception:
225-        pass
226-    return True
227-
--
319-    # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
320-    locale: Optional[str] = None
321-    # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
322:    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
323:    # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
324-    compute: Optional[str] = None
325-    # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
326-    # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
--
466-        "deployment": {
467-            "profile": getattr(deployment, "profile", "") or "",
468-            "compute_target": getattr(deployment, "compute_target", "") or "",
469:            # item 3: can this server satisfy a per-request compute="cloud" dispatch? The
SPA shows
470-            # the "run in the cloud" toggle only when true (else it's local-only).
471-            "cloud_available": _cloud_available(cfg),
472-        },
--
625-            if hasattr(config, "model_dump")
626-            else dict(config or {})
627-        )
628:    except Exception:
629-        cfg = {}
630-    report = dict(cfg.get("report") or {})
631-    if report.get("output_dir"):
--
690-        if not needs_proxy(src_local):
691-            return  # already light enough ? no proxy worth making
692-        make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4"))
693:    except Exception as e:  # noqa: BLE001 - warming is best-effort
694-        logger.warning("proxy: warm failed for %s: %s", video_id, e)
695-    finally:
696-        with _proxy_lock:
--
725-        )
726-    except ValueError as e:  # unsupported scheme etc. ? a clean client error, never a 500
727-        raise HTTPException(status_code=400, detail=str(e)) from e
728:    except Exception as e:  # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced (not
swallowed)
729-        raise HTTPException(status_code=502, detail=f"Could not resolve the source video:
{e}") from e
730-
731-    if input_ref.backend == "local":
--
735-            local = _cached_remote_source(video_id, input_ref, cfg)
736-        except HTTPException:
737-            raise
738:        except Exception as e:  # noqa: BLE001 - a fetch miss is surfaced (not swallowed)
739-            raise HTTPException(
740-                status_code=502, detail=f"Could not resolve the source video: {e}"
741-            ) from e
--
884-            )
```

```
885-        except ValueError as e:
886-            raise HTTPException(status_code=400, detail=str(e)) from e
887:        except Exception as e:  # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
888-            raise HTTPException(
889-                status_code=502, detail=f"could not resolve source_uri: {e}"
890-            ) from e
--
932-                    copied = True
933-                except ValueError as e:
934-                    raise HTTPException(status_code=400, detail=str(e)) from e
935:                except Exception as e:  # noqa: BLE001 ? a put failure is surfaced, never
silently dropped
936-                    logger.error(
937-                        "[ASSETS] put failed video_id=%s dest=%s: %s",
938-                        video_id,
--
1024-
1025-
1026-def _cloud_available(cfg) -> bool:
1027:    """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item 3).
1028-
1029-    True when the control plane is wired for Cloud Run: either the configured default
target is
1030-    already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
1031-    (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
1032:    are present so a forced per-request override can actually dispatch + collect a result.
1033-
1034-    Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it would
work."""
1035-    deployment = getattr(cfg, "deployment", None)
--
1042-    return bool(has_job and out_root)
1043-
1044-
1045:def _maybe_dispatch_remote(
1046-    cfg,
1047-    db,
1048-    req: ProcessRequest,
--
1058-    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
1059-
1060-    Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether the
cloud job
1061:    actually EXECUTED (``False`` for a pre-dispatch config rejection, so the observability
report
1062-    doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-process
segmenter
1063-    (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every non-``cloud_run``
target, so
1064-    the in-process path stays byte-identical)."""
--
1069-        # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
1070-        db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
1071-        # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run" only
when the
1072:        # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker rc
/ ingest
1073:        # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
1074:        # intended backend, `requested` the picker choice, `dispatched` whether the remote
job ran.
1075-        return (
```

```
1076-            {
1077-                "video_id": video_id,
--
1083-                    "path": "cloud_run" if ran else "not_run",
1084-                    "requested": req.compute,
1085-                    "target": "cloud_run",
1086:                    "dispatched": ran,
1087-                },
1088-            },
1089-            ran,
1090-        )
1091-
1092-    # SPA compute-picker (item 3): a per-request choice overrides the server's configured
default.
1093:    # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded
by
1094-    # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
1095-    choice = (req.compute or "").strip().lower()
1096-    if choice and choice not in ("local", "cloud"):
--
1139-            False,
1140-        )
1141-
1142:    notify("dispatching (cloud_run)")
1143:    logger.info(
1144:        "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
1145:        req.video_path,
1146:        out_root,
1147-    )
--
1160-        config_overrides=tuple(overrides),
1161-    )
1162-    try:
1163:        result = compute.run_infer(
1164-            spec
1165:        )  # dispatch the Cloud Run Job + bucket-poll to completion
1166:    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1167:        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
1168:        return _failed(f"cloud dispatch error: {e}", True)
1169:    if result.returncode != 0:
1170:        return _failed(f"cloud job failed (returncode {result.returncode}).", True)
1171:    if result.video_id and result.video_id != video_id:
--
1184-    )
1185-    try:
1186:        _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
1187:    except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not 500
or leak rows
1188:        logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
1189:        return _failed(
1190-            f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
--
1207-        "outputs_uri": result.outputs_uri,
1208-    }
1209:    logger.info(
1210:        "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s video_id=%s",
1211:        result.execution,
1212:        provenance["job"],
1213:        provenance["region"],
--
1269-            req.video_path, getattr(config, "storage", None)
1270-        )
1271:        video_id = compute_video_id(local_video)
```

```
1272:    except Exception:
1273-        storage.cleanup_acquired_local_input(
1274-            req.video_path, local_video, getattr(config, "storage", None)
1275-        )
1276-        raise
1277-    try:
1278-        existing_video = db.get_video(video_id)
1279:    except Exception:
1280-        storage.cleanup_acquired_local_input(
1281-            req.video_path, local_video, getattr(config, "storage", None)
1282-        )
--
1294-    val_context: Dict[str, Any] = {}
1295-    # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1296-    # "in_process" once the local segmenter is reached, set on the response as "cloud_run"
by
1297:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1298-    compute_actual: Optional[str] = None
1299-    response: Optional[Dict[str, Any]] = None
1300-    try:
--
1380-        # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter
below runs
1381-        # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer`
CLI yet, so
1382-        # they're dropped on the cloud path in this slice ? a documented follow-up.)
1383:        remote = _maybe_dispatch_remote(
1384-            config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
1385-        )
1386-        if remote is not None:
--
1395-            result = segmenter.process_video(
1396-                video_id, local_video, req.player_pool, req.max_frames
1397-            )
1398:        except Exception:
1399-            # A raising segmenter must not leave half-written rows: restore the pre-run
1400-            # state (same destroy-after-confirm contract as the Failure branch), then let
1401-            # the job worker record the failure.
--
1442-            )
1443-        prov = response.get("compute", {}) if isinstance(response, dict) else {}
1444-        logger.info(
1445:            "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
1446-            video_id,
1447-            getattr(req, "compute", None),
1448-            prov.get("path"),
1449:            prov.get("dispatched"),
1450-            (response or {}).get("status") if isinstance(response, dict) else None,
1451-        )
1452-        # Always emit the per-video observability report (next to the video, or to
--
1490-        )
1491-        backend = storage.get_storage(input_ref.backend, config=sc)
1492-        return int(backend.stat(input_ref).size)
1493:    except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must not
block
1494-        logger.debug(
1495-            "free-space: could not stat remote source %s: %s", input_ref.uri, e
1496-        )
--
1831-            video["filepath"], getattr(cfg, "storage", None)
1832-        )
1833-        out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
```

```
1834:    except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1835-        logger.error(
1836-            "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1837-        )
--
1977-            logger.info(
1978-                "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
1979-            )
1980:    except Exception as e:
1981-        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1982-
1983-
--
2102-                args.video_path, [(args.trim_start, args.trim_end)], out_name
2103-            )
2104-            print(f"[SUCCESS] Trimmed segment saved to: {res_path}")

[Showing results with pagination = limit: 250]
```

19. assistant (2026-07-02T08:17:43.717Z)

The caller at `backend/main.py:1162-1168` wraps `compute.run_infer(spec)` in `except Exception as e:` (broad catch, BLE001) and turns any dispatch/poll error into a clean `_failed(...)` job failure. So the PR's re-raised `PolicyTimeout` (a `TimeoutError` subclass) is still caught by this handler ? the behavior contract holds. The PR's honest failure message (job/execution name + cancel outcome) will flow into `_failed`'s message and get logged. Good ? no caller change needed, and no regression.

Let me verify the test files' merge cleanliness for the two touched test files (the modern merge-tree already produced a clean combined tree `bf5fc42`, but let me confirm the two test files have no conflict there specifically) and confirm what #427 changed in `test_compute_backend.py`.

20. user (2026-07-02T08:17:52.424Z)

```
=== Did #427 touch the two test files PR #417 edits? ===
19ebe5a clean remote input scratch dirs (#427)
=== (empty above = master hasn't moved these test files since PR merge-base) ===

=== But merge-tree said test_compute_backend changed in both. Check what commit on master
touched it AFTER PR branch point via full range ===
19ebe5a clean remote input scratch dirs (#427)
=== confirm final merged tree has no conflict blobs (grep merged test file) ===
0
^ conflict-marker count in merged test file (0 = clean)
```

21. assistant (2026-07-02T08:18:00.149Z)

All confirmed. #427 touched `test_compute_backend.py` too, but the modern 3-way merge (`bf5fc42`) has **0 conflict markers** in the merged test file ? non-overlapping hunks. The merge is genuinely clean.

Let me do one final check on the correctness of the PR change itself: verify the `_read_marker` returns `None` when absent (so the "final marker check" logic ? `late = self._read_marker(...)` returning `None` ? treated as absent ? cancel) is correct, and that `_read_marker` can return an empty dict `{}` (present-but-empty) which the PR's `if late is not None` would rescue. That's a subtle correctness point worth noting.

Looking at master `_read_marker` (lines 204-211): it returns `None` when the marker doesn't exist, or `self._read_json(done_uri) or {}` (an empty dict) when present. The PR's `_handle_poll_timeout` does `if late is not None: return late` ? so a present-but-empty marker (`{}`) is rescued and returned. That empty dict then flows into `run_infer`'s existing empty-marker handling (lines 179-189: `video_id=""` ? rc defaults to 1 ? treated as failure). This is

consistent/correct behavior ? a present marker means the worker finished, and the empty-marker path correctly treats it as a failure result rather than a timeout. No bug.

I have everything needed. Let me record my findings.

22. user (2026-07-02T08:18:24.824Z)

Structured output provided successfully